

Rust dans le noyau Linux

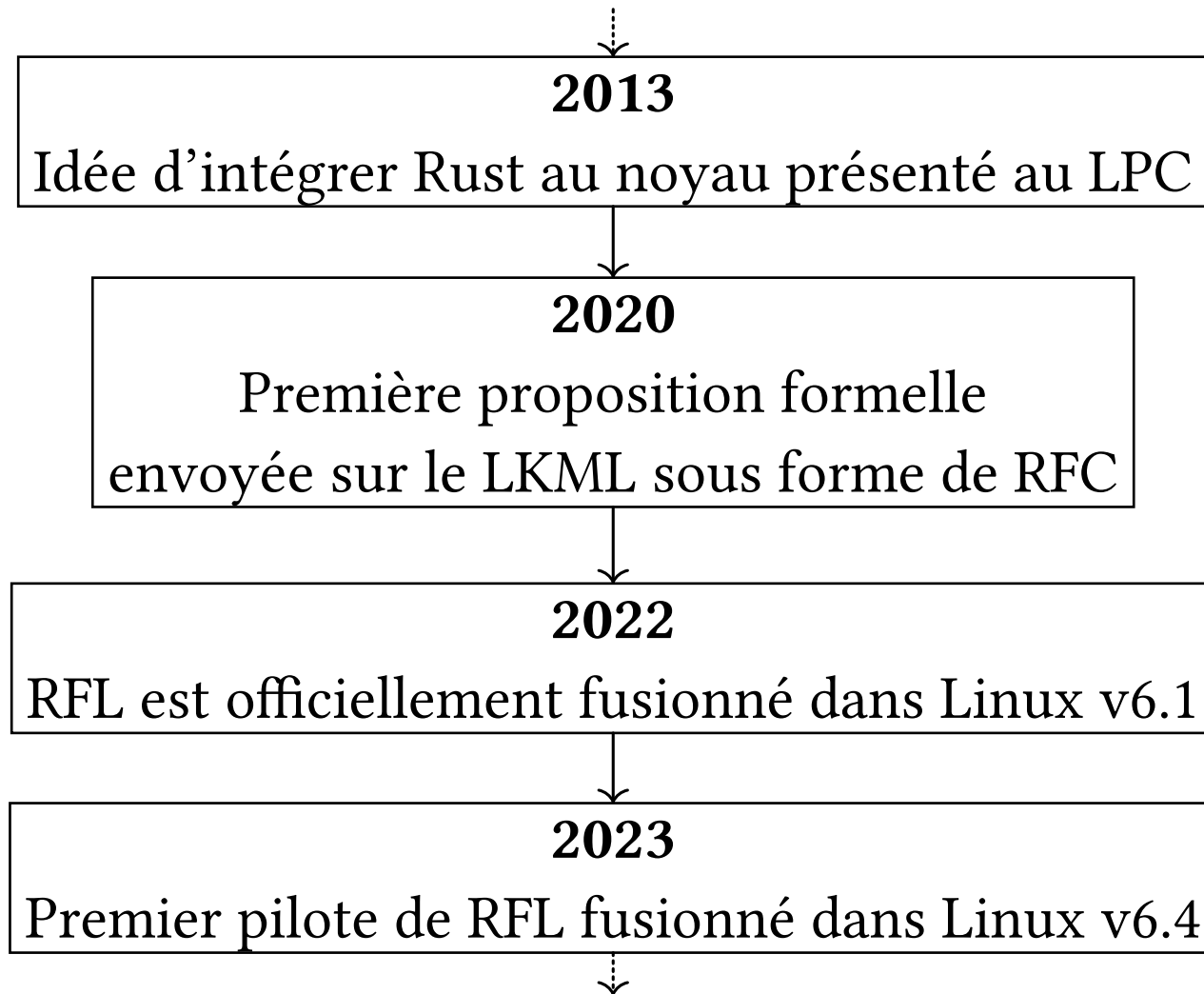
Fares Mestre

Dudillieu Gabin

Introduction

- Le noyau Linux ne cesse d'éliminer des bugs mémoires et de concurrences.
- Des bugs sont encore retrouvés :
 - Problème qui affectait les fonction *tcp_getsockopt/tcp_setsockopt* du composant *TCP HANDLER* dont la manipulation entraînait une condition de concurrence.
 - Problème qui affectait la fonction *bq24190_remove* dans code de *drivers/power/supply/bq24190_charger.c*, qui pouvait permettre à un attaquant local de bloquer le système en raison d'une condition de concurrence.

Histoire RFL



Objectifs

Domaine	Objectifs
Sécurité	Pilote avec mémoire et threads sûr
Performance	Aucune surcharge liée aux abstractions
Outils	Meilleure documentation et des bons tests d'intégrations
Efficacité	Meilleure efficacité de développement
Communauté	Plus de développeurs pour le noyau

Comment Rust garantit sa sûreté

- 3 mécanismes principaux
 - Des portées propriétaires de ses variables
 - L'emprunt des variables et la mutabilité explicite
 - La durée de vie de l'emprunt
- Ces vérifications se font à la compilation!

Ownership

```
fn print(s : String) {  
    println!("{}", s)  
}
```

```
fn main() {  
    let s = String::from("Hello");  
    print(s);  
    print(s);  
}
```

Ownership

```
fn print(s : String) {  
    println!("{}", s)  
}
```

```
fn main() {  
    let s = String::from("Hello");  
    print(s);  
    print(s);  
}
```

s est utilisable dans cette portée

Ownership

```
fn print(s : String) {  
    println!("{}", s)  
}
```

```
fn main() {  
    let s = String::from("Hello");  
    print(s);  
    print(s);  
}
```

mais s se déplace...

Ownership

```
fn print(s : String) {  
    println!("{}", s)  
}
```

```
fn main() {  
    let s = String::from("Hello");  
    print(s);  
    print(s);  
}
```

s n'est plus disponible!

Ownership

```
fn print(s : &String) {  
    println!("{}", s)  
}
```

```
fn main() {  
    let s = String::from("Hello");  
    print(&s);  
    print(&s);  
}
```

Solution : Borrows

s ne se déplace plus

Sortie :

Hello

Hello

Mutabilité

- On souhaite implémenter la gestion du contenu des fichiers

```
struct FileTableEntry {  
    inode: u32,  
    modes: u8,  
    pos: u64  
}
```

- On veut implémenter un seek qui nous permettra de changer la position de la tête de lecture

Mutabilité

```
fn seek(&self, offset: i32) {  
    let mut subtract = false;  
    if offset < 0 {  
        offset = -offset;  
        subtract = true;  
    }  
    let offset = offset as u64;  
    if subtract {  
        self.pos -= offset  
    } else {  
        self.pos += offset  
    }  
}
```

Ce code est incorrect à plusieurs endroits

Mutabilité

```
fn seek(&self, offset: i32) {  
    let mut subtract = false;  
    if offset < 0 {  
        offset = -offset;  
        subtract = true;  
    }  
    let offset = offset as u64;  
    if subtract {  
        self.pos -= offset  
    } else {  
        self.pos += offset  
    }  
}
```

offset n'est pas mutable

self n'est pas mutable

Mutabilité

```
fn seek(&mut self, offset: i32) {  
    let mut subtract = false;  
    let offset = if offset < 0 {  
        subtract = true ;  
        -offset as u64  
    } else {  
        offset as u64  
    };  
    if subtract {  
        self.position -= offset  
    } else {  
        self.position += offset  
    }  
}
```

self est un **unique borrow**

Lifetimes

- On veut maintenant modéliser des processus

```
struct Proc {  
    pid: u32,  
    ppid: u32,  
    status: char,  
    fd_vec: Vec<u8>,  
    prog: fn() -> i8  
}
```

- On implémente (très naïvement) une methode pour vérifier lequel de deux processus est le plus ancien

Lifetimes

```
fn eldest<'a>(&'a self,  
  other : &'a Proc) -> &'a Proc {  
    if self.pid == other.ppid {  
        self  
    } else if other.pid == self.ppid {  
        other  
    } else {  
        if self.pid > other.pid {  
            other  
        } else {  
            self  
        }  
    }  
}
```

La durée de vie doit être explicitée ici

Lifetimes

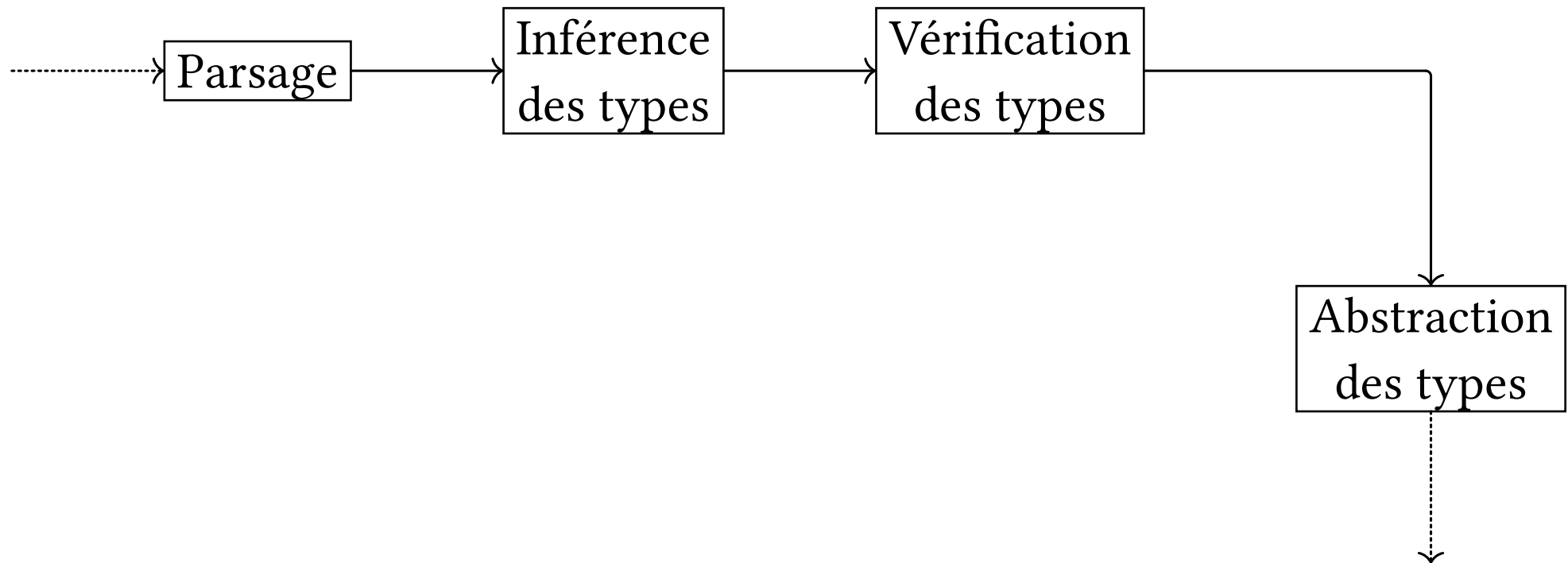
```
fn main() {  
    let init_proc = /* ... */;  
    let parent; {  
        let child =  
            init_proc.fork(fresh_pid());  
        parent =  
            init_proc.eldest(&child);  
        print!("old : {}\\n", parent.pid)  
    }  
    print!("old : {}", parent.pid)  
}
```

parent hérite du lifetime
le plus court

parent est périmé!

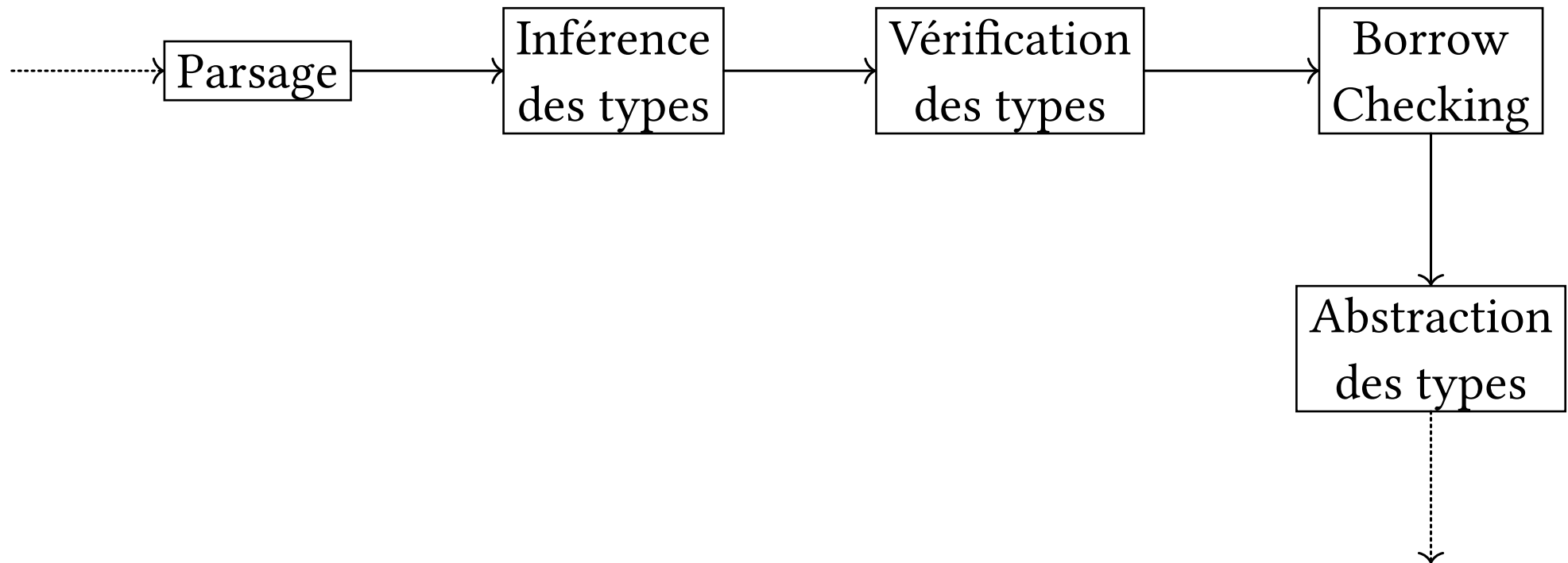
Le Borrow Checker

- Un processus de compilation toplevel standard :



Le Borrow Checker

- Il y a une étape de plus en Rust



Le Borrow Checker

- Fonctionne sur une représentation en Graphe de Contrôle du Flot (GCF) de Rust
- Trois étapes pour une variable de durée de vie 'a
 - Calcul des contraintes sur l'ensemble des noeuds associés à 'a
 - Calcul des ensembles minimaux associés à 'a
 - Vérification des accès
 - Aucun accès s'il existe un autre prêt mutable vivant
 - Aucune lecture si la variable n'est pas initialisée ou déplacée ailleurs
 - Aucune écriture s'il existe un autre prêt immutable vivant

Mais ces garanties ont un coût...

Unsafe

```
fn main() {  
    let init_proc = /* ... */;  
    let parent; {  
        let child =  
            init_proc.fork(fresh_pid());  
        parent =  
            init_proc.eldest(&child);  
        print!("old : {}\\n", parent.pid)  
    }  
    print!("old : {}", parent.pid)  
}
```

comment est implémenté
fresh_pid ?

Unsafe

```
static mut NEXT_PID : u32 = 2;  
fn fresh_pid() -> u32 {  
    let f = NEXT_PID;  
    NEXT_PID += 1;  
    f  
}
```

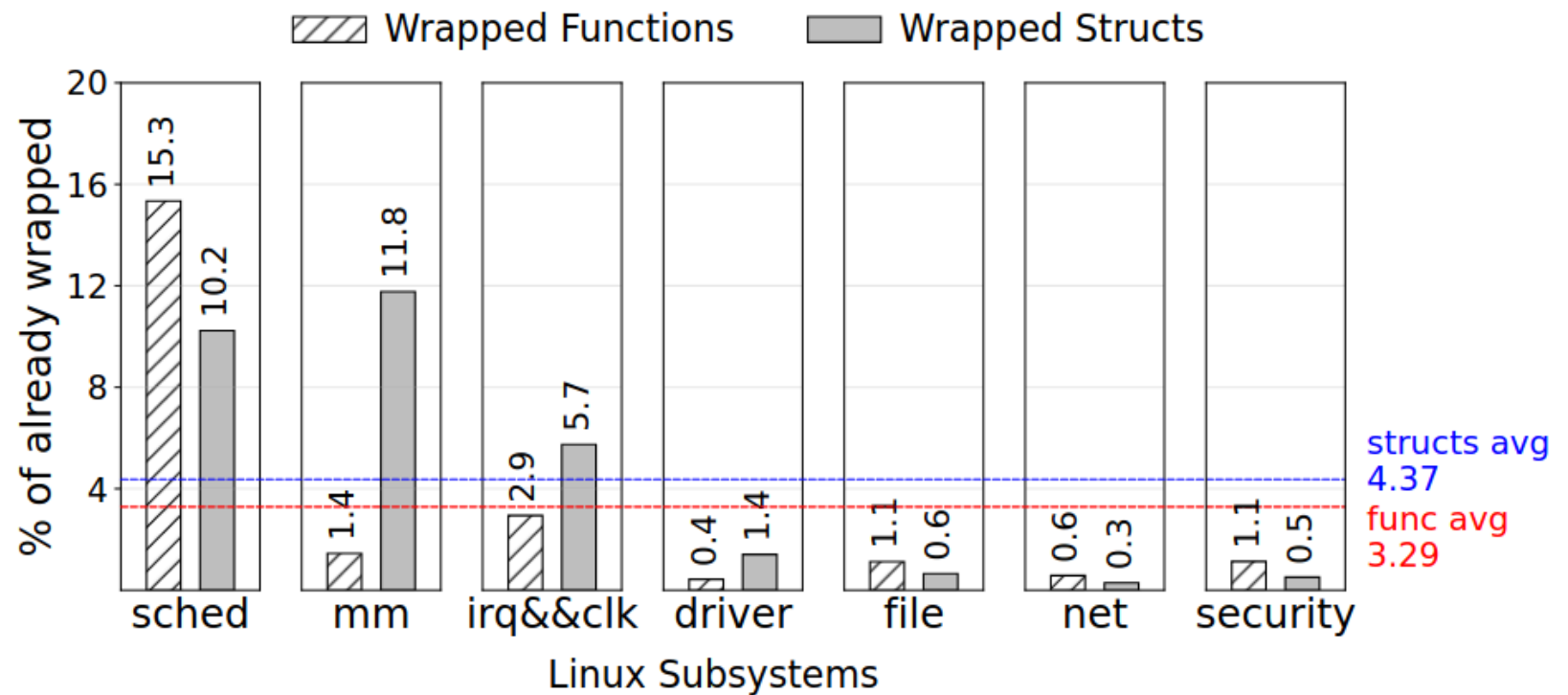
La manipulation de variables statiques n'est pas garantie sécurisée!

Il faut marquer `fresh_pid` comme `unsafe`!

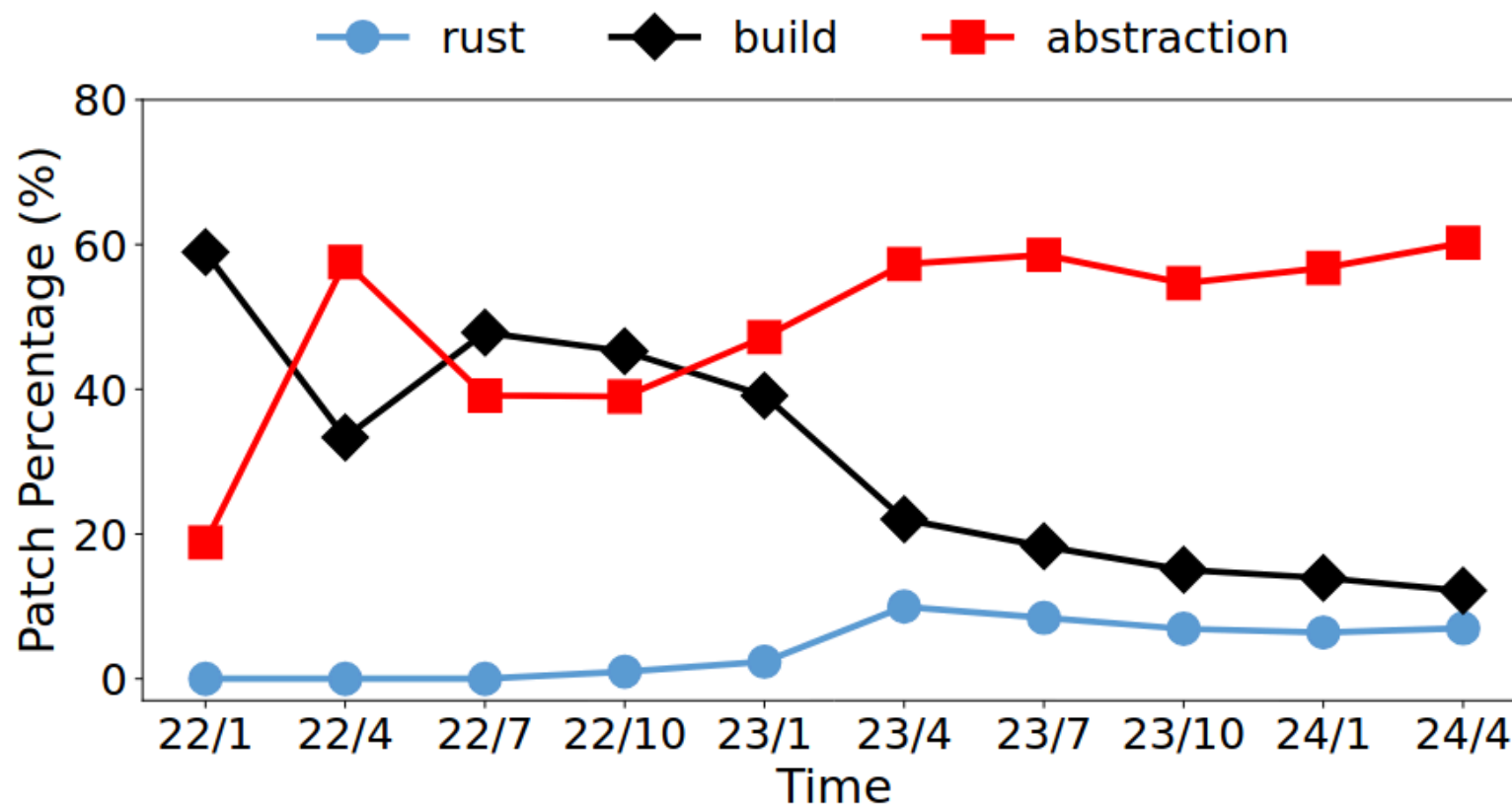
Réalisation

- Plus de 160 commits officiellement ajouté dans le noyau (19K lignes de code).
- Plus de 1300 commits en attente avant intégration officielle (112K lignes de codes).
- Plus de 500 commits en attente de vérification (186K lignes de code).

Evolution des APIs Rust de RFL dans le temps



Répartition des patches Rust de RFL dans le temps



Difficultés : Intéropérabilité avec le noyau

- Pas de std ni libc.
- Ne peut pas utiliser les allocateurs système classique de Rust (new).

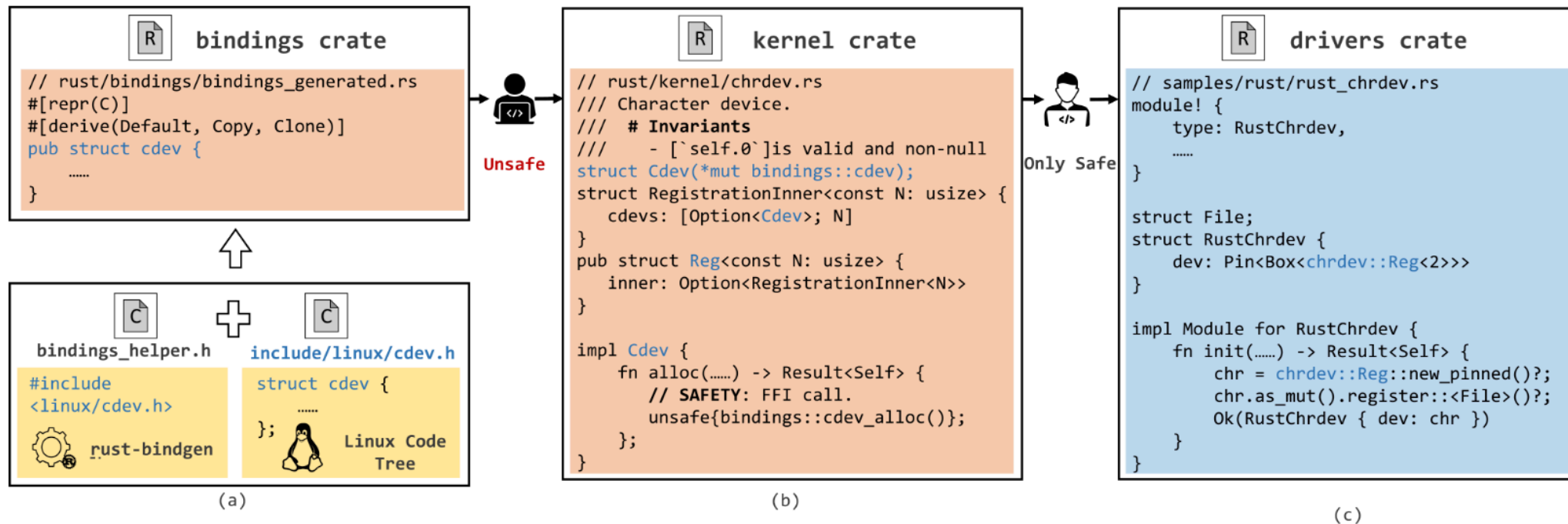
Solution

- Utiliser le système d'allocation mémoire (kmalloc) avec ses contraintes (atomicité dans certains contextes, invariants, panic, ...).
- Réimplémenter *GlobalAlloc* de Rust.

Nouvelles difficultés

- Les fonctions que Rust veut utiliser sont dans des headers C du noyau.
- Il faut rendre le code C et Rust intéropérable.

Architecture RFL



Difficultés : Développement des abstractions sécurisées

- Problème : Les conventions de programmation de C et Rust sont bien différentes.
 - C utilisent beaucoup le transtypage, l'arithmétique des pointeurs, les opérations sur les bits.
 - Ces conventions contredisent les règles de sécurité de Rust.

Règles de traduction de Rust-Bindgen

Type	C	Rust
Types primitifs	foo	core::ffi::c_foo
Types pointeurs	foo*	*mut foo
Attribut	aligned	#repr(c)
Pointeurs de fonctions	fn	option<fn>

Mais pas tout les types C sont traduisibles en une primitive Rust correspondante.

Exemple champs de bits en C

```
struct bits_field {  
    unsigned int a : 1; // bit 0  
    unsigned int b : 1; // bit 1  
    unsigned int cd : 2; // bits 2-3  
    unsigned int ez : 28; // bits restants  
};
```

```
struct bits_bf;  
bf.a = 1;  
bf.b = 2;
```

Exemple traduction champs de bits en Rust

```
struct BitsField {  
    data: [u8; 4],  
}
```

```
impl BitsField {  
    fn a(&self) -> bool {  
        self.data[0] & 0b00000001 != 0  
    }  
}
```

```
fn set_a(&mut self, v: bool) {  
    if v {  
        self.data[0] |= 0b00000001;  
    } else {  
        self.data[0] &= !0b00000001;  
    }  
}
```

```
}

fn b(&self) -> u8 {
    (self.data[0] & 0b00001100) >> 2
}

fn set_b(&mut self, v: u8) {
    self.data[0] = (self.data[0] & !
0b00001100) | ((v & 0b11) << 2);
}
}
```


Difficultés : Développement des pilotes

- Problème : Aussi impacté par ces différences de paradigmes.
 - Code plus complexe.
 - Code plus verbeux.

Exemples

- Chaque type doit être annoté avec :
 - Arc si il va être partagé entre threads.
 - Pin si il va rester immuable en mémoire.
- Ces annotations peuvent être imbriquées et plus complexe comme le buffer anneau rx du driver e1000 qui devient `Pin<Box<SpinLock<Box<Ring<RxDesc>>>>>`.

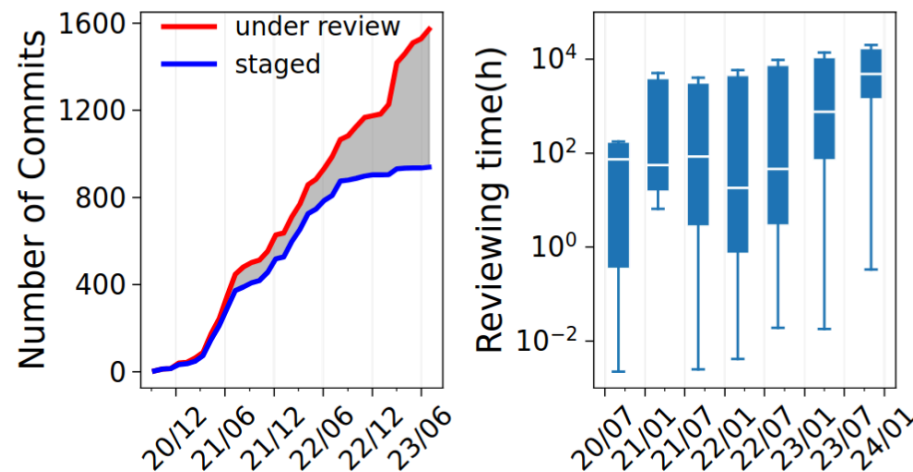
Autres exemples

- Tableaux dynamiques
 - C utilise des pointeurs simples.
 - Rust exige des couches supplémentaires et des traits, qui augmente le nombre de ligne de codes.
- Contextes d'exécution
 - Non vérifiés par les règles de sécurité de Rust (atomique, sleepable, ...).
 - Le développeur doit lui-même déterminer le contexte d'exécution de la fonction appelante et devient responsable de la sécurité des threads
- Nettoyage du périphérique
 - En C, un goto qui centralise le nettoyage.
 - En Rust, un trait Drop et une couche d'abstraction sont ajoutée pour automatiser le recyclage de ressource.

Difficultés : Jeunesse du projet

- Problème : Le projet est encore jeune.
 - Des drivers et systèmes de fichiers manquent.
 - Manque de reviewers qualifiés.
 - Conflits entre développeurs.

Tendance des commits et reviews de RFL



Résultats : Sécurité

Depuis 2024 :

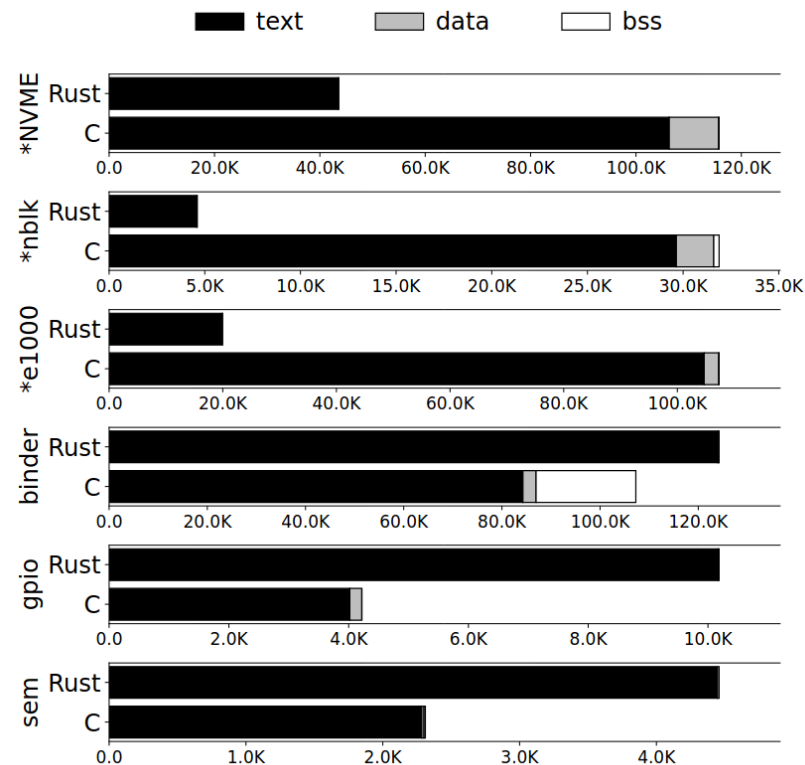
- 25 bugs trouvés dans RFL.
- 15 dans la branche principale Linux et 10 dans celle se préparant à être ajoutées.
- Parmi ceux de la branche principale, 11 sont des bugs de compilations.
- Les 4 autres sont liées à l'abstraction sécurisées.
- Parmi tous, 6 peuvent compromettre la mémoire.
- Parmi tous, 3 peuvent compromettre la sécurité des threads.

Principales causes des problèmes de sécurité

- Unsafe dans le code Rust
 - Mais sont parfois inévitable
 - Soit parce que safe rajouterait trop de complexité.
 - Soit les conditions de sécurité de Rust sont trop contraignantes.
- Rust n'élimine pas toutes les vulnérabilités
 - Parmi 240 vulnérabilités étudiées :
 - 82 sont éliminées automatiquement par Rust (user-after-free, buffer overflow, ...)
 - 112 requiert des pratiques spécifiques de rust (gestion de Drop, vérifications explicites, ...)
 - 45 non affectées par Rust (erreurs protocolaires, sémantique, ...)

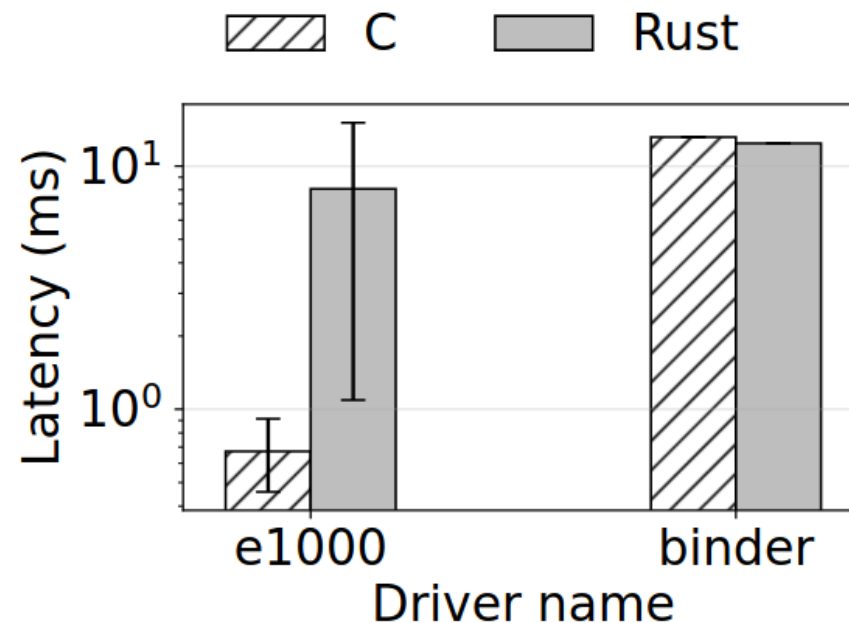
Résultats : Efficacité (Taille)

La taille des binaires sont la majeure partie du temps plus volumineux que les équivalents en C (environ 33% de codes en plus).



Résultats : Efficacité (Performance)

Performances comparées à C qui varie en fonction du driver (vérifications, locks pour la sécurité des threads, ...).



Résultats : Efficacité (Performance)

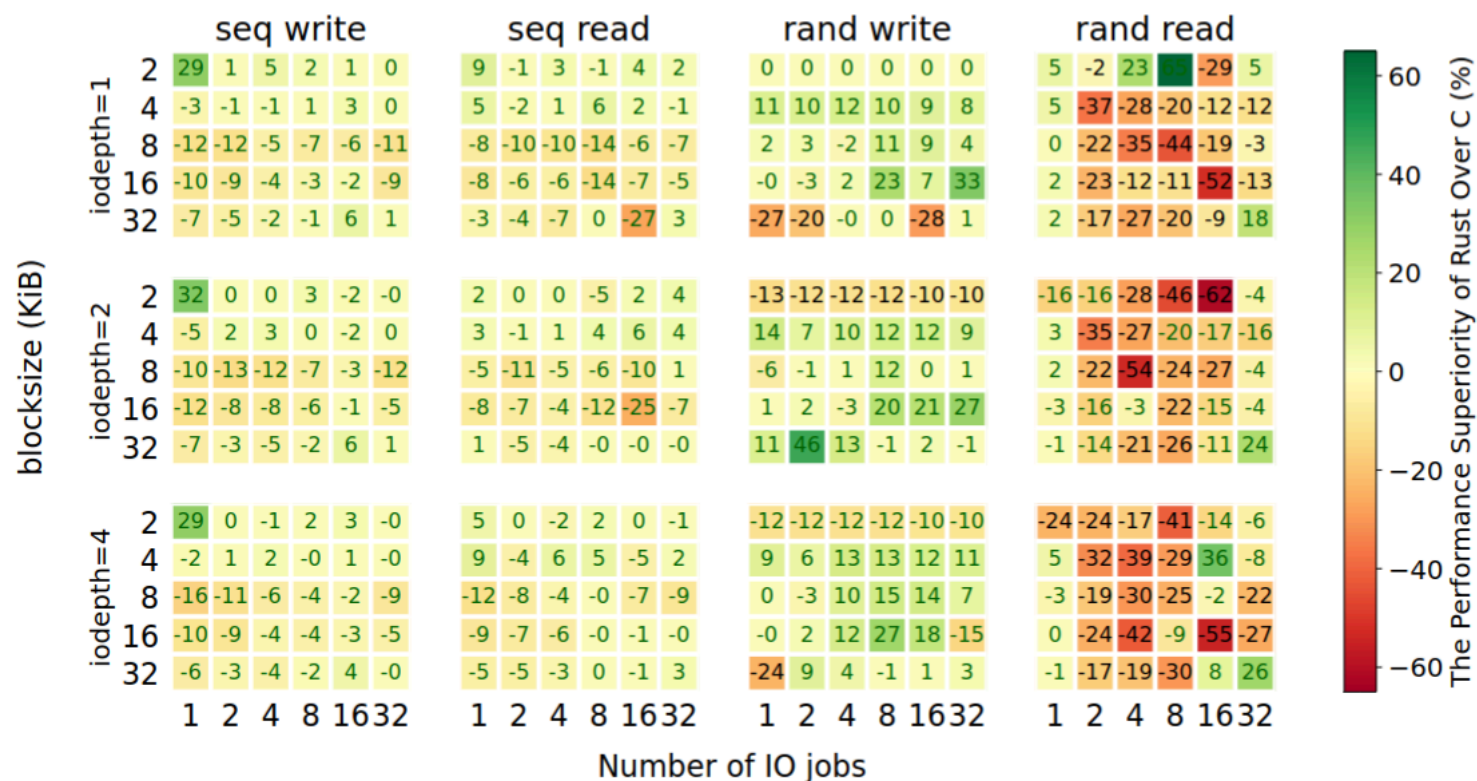


Figure 7: Comparaison de performance du pilote NVME

Resultats : Développement

- Méthodologie de développement particulièrement rigoureuse.
- Fort accent sur la documentation (100% du code) et justifications des abstractions unsafe.
- Gestion des bugs structurée et exigeante
 - CI pour vérifier le code
 - Patches intensément review
 - Erreurs traitées systématiquement
- Travail communautaire qui se renforce.

Conclusion

- RFL n'est pas parfait.
- Constitue un bon exemple d'intégration d'un nouveau langage dans un système mature.

Bibliography

- [1] H. Li, L. Guo, Y. Yang, S. Wang, and M. Xu, “An empirical study of {Rust-for-Linux}: The success, dissatisfaction, and compromise”, in *2024 USENIX Annual Technical Conference (USENIX ATC 24)*, 2024, pp. 425–443.
- [2] Z. Li, V. Narayanan, X. Chen, J. Zhang, and A. Burtsev, “Rust for Linux: Understanding the security impact of Rust in the Linux kernel,” in *2024 Annual Computer Security Applications Conference (ACSAC)*, 2024, pp. 548–562.
- [3] cvedetails.com, “Linux Kernel TCP tcp_setsockopt race condition.” [Online]. Available: <https://www.cvedetails.com/cve/CVE-2022-3566/>

- [4] cvedetails.com, “Use-After-Free Race Condition Local Denial of Service Vulnerability.” [Online]. Available: <https://www.cvedetails.com/cve/CVE-2023-33288/>
- [5] Steve Klabnik Carol Nichols and w. c. f. t. R. C. Chris Krycho, “Rust Book.” [Online]. Available: <https://doc.rust-lang.org/book/>
- [6] L. K. dev community, “Linux Rust Kernel Documentation.” [Online]. Available: <https://docs.kernel.org/rust/index.html>
- [7] rust-lang, “rust-bindgen.” [Online]. Available: <https://github.com/rust-lang/rust-bindgen>
- [8] J.-H. Jourdan, “Introduction to Rust.” [Online]. Available: <https://gitlab.inria.fr/fpottier/mpri-2.4-public/-/blob/master/slides/jhjourdan-01.pdf>